

FalconVQA

Fast Augmented Language-based CONversational
Video Question Answering

A Semantic Retrieval Layer for Surveillance Footage

Consolidated Report: Phases 1 to 4

Avinash Anish

Vaivaswat Verma

August 6, 2026

FalconVQA is a semantic retrieval layer for recorded video. A recording is partitioned into chunks, each chunk is analysed by vision and audio models, the results are indexed and rolled up to video level, and the whole is then searched or questioned in natural language. Every result carries a timecode that plays. This document consolidates the four project phases: the problem and its plan, the detailed design and acceptance plan, the implementation, and the delivered product with its measured behaviour.

Contents

I	Phase 1: Problem, Architecture and Planning	2
1	Problem Statement	2
1.1	System Identification	2
1.2	Statement	3
1.3	Background	3
1.4	Why Existing Tools Do Not Solve It	3
1.5	Objectives	3
1.6	Requirements	4
2	Creative Enhancement	5
3	High-Level Architecture	7
3.1	Tier Structure	7
3.2	The Processing Pipeline	8
3.3	Ingestion Lifecycle	8
3.4	Technology Stack	8
4	UI and Acceptance Planning	9
4.1	Interface Plan	9
4.2	Acceptance Planning	10
5	Execution Plan	11
II	Phase 2: Detailed Design, Interface, Dependencies and Test Plan	11
6	Detailed Design	12
6.1	Design Flow	12
6.2	Database Design	16
6.3	Normalisation	17
6.4	Schema	17

6.5	Modularity	19
6.6	Scalability	20
6.7	Security	20
7	Detailed User Interface	21
8	Third-Party Libraries and Tools	23
8.1	Extent of the Dependency	23
8.2	Principal Dependencies	23
8.3	Paid and Metered Services	25
9	Acceptance Test Plan and Test Cases	25
9.1	Plan	25
9.2	Test Cases	26
9.3	Test Automation	29
10	Documentation of Process, Design, ATP and ATCs	30
III	Phase 3: Implementation	30
11	Coding Standards	30
12	User Implementation	31
13	Scalable and Sustainable Design	31
14	Dataset Creation	32
IV	Phase 4: Product, Performance and Delivery	33
15	Product Demo: Completeness	33
16	Product Demo: Correctness	34
17	Product Performance	34
17.1	Precision	34
17.2	Latency	35
17.3	Memory and Processor Usage	36
17.4	Scale Limits	39
18	UI Implementation	39
19	Documentation	39

I Phase 1: Problem, Architecture and Planning

1 Problem Statement

1.1 System Identification

FalconVQA stands for Fast Augmented Language-based CONversational Video Question Answering. It is a retrieval and question answering system for recorded video, developed with surveillance footage as its primary application domain. The name records the four properties the system is built

around: retrieval fast enough to be interactive, an index augmented beyond the raw signal with structured descriptions produced by vision and audio models, a language-based interface in which both the index and the query are natural language, and a conversational access path in which an operator or an autonomous agent asks questions of a recording and receives cited answers rather than a list of files.

1.2 Statement

A natural language front end that extracts the relevant footage from a given recorded video by description alone.

1.3 Background

Surveillance footage is recorded onto network video recorders and other digital storage devices. Finding a particular piece of footage is carried out by entering the date and time of the incident and reviewing what comes back. The recording exists as a continuous timeline indexed by nothing but a clock, so the date and time are the only handles the operator has.

That procedure works only when the time of the incident is already known. In practice it usually is not. What is available instead is a description of the event, obtained from a witness or from a report: a man in a yellow shirt left a bag near the checkout. No stage of the recording chain produces any account of what the footage contains, so a description of this kind cannot be used as a query at all.

1.4 Why Existing Tools Do Not Solve It

Two things stand between the description and the playback. The first is that the footage carries no description to match against, so one has to be derived. The second is that general-purpose video understanding tools do not derive it, because they are built for edited media and rely on three properties surveillance footage does not have.

- **Shot boundaries.** A fixed camera produces none.
- **Speech.** A substantial proportion of surveillance footage is silent.
- **Visually salient events.** Events of interest are typically brief and visually unremarkable.

Applied to such footage, these tools either return the entire recording as a single segment or subdivide it into a large number of near-identical segments. Neither result is searchable. The problem is therefore twofold: derive a descriptive representation of footage that has little intrinsic structure, and make the footage retrievable through that representation.

1.5 Objectives

- O-1. A processing pipeline that accepts a recording and renders it searchable by description.
- O-2. Semantic search returning ranked moments with timecodes, and question answering that returns a synthesised answer with references to the moments it was derived from.
- O-3. Video-level output that can be read in place of watching the footage: a summary, chapters, notable events, the people and objects that appear, and any anomalies.
- O-4. A multi-tenant web application in which uploading, searching, questioning and playback are performed in a single workspace, with each operator's recordings isolated from every other operator's.
- O-5. A documented and self-describing HTTP API, so that the same capabilities can be driven by an LLM agent rather than only by a human operator.

1.6 Requirements

Functional requirements are stated from the operator’s perspective. They are implemented across the client application and the core analysis service.

ID	Capability	The system shall allow the operator to...
FR-1	Account and workspace	Register, authenticate, and organise recordings into named projects, with each operator able to reach only their own projects and recordings.
FR-2	Adding footage	Add a recording to a project, either as an uploaded file or as a link, and select how thoroughly it is to be examined, trading processing cost against depth of detail.
FR-3	Tracking progress	Observe that a recording is being processed, which stage it has reached and how far it has advanced, and continue working while processing proceeds.
FR-4	Searching	Locate moments by describing what occurred in natural language, without knowing when it occurred or what it was called.
FR-5	Narrowing results	Restrict a search to a period of the recording, to footage containing particular people, objects or spoken content, or to a chosen subset of recordings.
FR-6	Asking questions	Pose a question about one or more recordings and receive a direct answer accompanied by references to the moments from which it was derived.
FR-7	Reviewing results	Determine exactly when each result occurred and play the footage from that moment, individually or as a continuous reel.
FR-8	Understanding a recording	Review a recording as a whole without watching it: summary, chapters, notable events, the people and objects that appear and when, and any anomalies.
FR-9	Conversing about footage	Conduct a multi-turn conversation in which the assistant selects the appropriate retrieval operations, and have that conversation persist for later resumption.
FR-10	Inspecting the index	Examine what the system recorded for any individual chunk alongside a time-line synchronised with playback.
FR-11	Re-examining footage	Re-examine a recording already held at a different depth of analysis, without re-adding it and without discarding results already obtained.
FR-12	Programmatic access	Drive every retrieval capability over a documented HTTP API whose analyzers, filters and response shapes are discoverable from the API itself.

Table 1: Functional requirements.

Non-functional requirements are stated as service targets an operator would observe. The figures below are design budgets confirmed by the measurements in Section 17.

ID	Quality	Requirement
NFR-1	Responsiveness	A description search returns ranked moments within 2 seconds; a question is answered within 15 seconds.
NFR-2	Processing time	A recording becomes searchable within approximately three times its own duration, and the operator may continue working meanwhile.
NFR-3	Transparency	Processing stage and progress remain visible throughout ingestion, refreshed at least every 5 seconds.
NFR-4	Playback latency	Selecting a result begins playback at that moment within 2 seconds, with no manual scrubbing.
NFR-5	Traceability	Every answer cites the moments it was derived from, and the system states that the footage does not support an answer rather than producing one.
NFR-6	Robustness	Unedited, frequently silent, fixed-camera footage is handled as the normal case rather than as a failure case.
NFR-7	Cost control	The operator selects how thoroughly each recording is examined, and reviewing results already obtained incurs no further processing cost.
NFR-8	Extensibility	A new analyzer or aggregator is added as a single module and one registry entry, with no modification to ingestion, storage or the API.
NFR-9	Data isolation	Recordings, projects and conversations are readable only by the account that owns them, enforced at the database and at the agent boundary.
NFR-10	Idempotent ingestion	A recording is identified by the content of its bytes, so re-submitting the same file is recognised as the same recording.

Table 2: Non-functional requirements.

2 Creative Enhancement

The enhancements below were, in most cases, adopted in response to a measured failure on real footage. Collectively they are what distinguishes the system from a conventional partition-the-video-and-embed-the-captions pipeline. Each is annotated with the requirement it primarily serves.

#	Enhancement	Rationale	Serves
E-1	Multi-signal chunking with weight redistribution	Four detectors contribute evidence for each cut, and a signal that produces no events cedes its weight to the signals that did. Without this, fixed-camera footage is returned as a single chunk, and a preset degenerates into a granularity dial: on single-signal footage the <code>audio</code> and <code>video</code> presets yielded 12 and 63 boundaries from identical content.	NFR-6
E-2	Concurrent chunkings of one recording	The chunk configuration is stored with the records, so a recording may be indexed coarsely and finely at once and a query resolves against whichever is appropriate.	FR-11
E-3	Five named vectors per chunk	Description, people, actions and objects are embedded separately alongside the combined record, so a query about a person’s appearance is matched against a short person description rather than competing with everything else recorded for that chunk.	FR-4
E-4	Inexpensive detectors used as gates	YOLO and EasyOCR determine whether a frame merits submission to the vision-language model. Frames containing nothing to see or read incur no model cost.	NFR-7
E-5	Detector labels withheld from the vision model	YOLO classified a thermally imaged tank as an airplane with 0.73 confidence. Shown the same bounding boxes without labels, the vision model reported a tank. A label rendered onto the image invites agreement with it.	NFR-5
E-6	Adaptive frame sampling	Frames are retained according to their dissimilarity from the last frame retained. A fixed threshold of 0.95 retained one frame on a static camera, whose consecutive similarity median was 0.9955; a purely relative threshold selected compression noise. Both were measured.	NFR-2
E-7	Questions routed to aggregates	A question is directed to the video-level output capable of answering it rather than searching segments and synthesising from whatever is returned.	FR-6
E-8	Constrained entity linking	People are clustered on appearance under hard constraints, chief among them that two people visible in the same chunk cannot be the same person, before an LLM writes any narrative. Asked to match people directly, an LLM merges anyone in dark clothing.	FR-8
E-9	Object linking with fixture suppression	Object entities reuse the same clusterer with two filters: static fixtures such as counters and shelves are counted but not tracked, and person-like detections are handed to the people aggregator, which has clothing to identify them with.	FR-8
E-10	Selectable response detail	The same search returns a few hundred tokens or tens of thousands. Five full hits were measured at approximately 19,000 tokens against approximately 440 at <code>minimal</code> . A human reader wants the whole record; an agent pays per token.	NFR-7
E-11	Self-describing API	One endpoint returns every analyzer, vector field, filter and aggregator the installation supports, so a client or an LLM agent discovers what may be asked instead of being hard-coded against it.	FR-12, NFR-8
E-12	Content-addressed recordings	A recording is identified by the hash of its bytes, so the same file submitted twice is neither processed nor stored again. No local path is ever persisted.	NFR-10
E-13	Registry-driven composition	Analyzers and aggregators are declared in registries and discovered from them. An aggregator whose analyzer is absent is skipped rather than failed.	NFR-8
E-14	Project-scoped agent tools	The analysis service has no notion of users, so every agent tool resolves the recordings it may reach through a single scoping function bound to the caller’s project. A fabricated video identifier is filtered out rather than answered from another operator’s footage.	NFR-9
E-15	Job reconciliation with lost-job recovery	Ingestion returns immediately and proceeds on a background thread. One reconciliation routine converts a job into a database row for every route that needs it, and a restart mid-ingest is detected and recovered or failed explicitly.	NFR-3
E-16	Clips as ranges, not files	A clip is a time range ⁶ over one recording rather than a stream of its own, so several moments from one recording share a single media load and no clip is rendered.	NFR-4
E-17	Poster frame written	One JPEG is written per recording at a fixed fraction of its du-	NFR-1

3 High-Level Architecture

3.1 Tier Structure

The system is a three-tier application. A Next.js client provides the operator interface and hosts an LLM agent that translates conversational requests into retrieval calls. A FastAPI service owns the entire video pipeline and exposes it as a self-describing HTTP API. Persistence is divided across object storage for media, a relational database for application state, a document store for analysis output, and a vector database for retrieval.

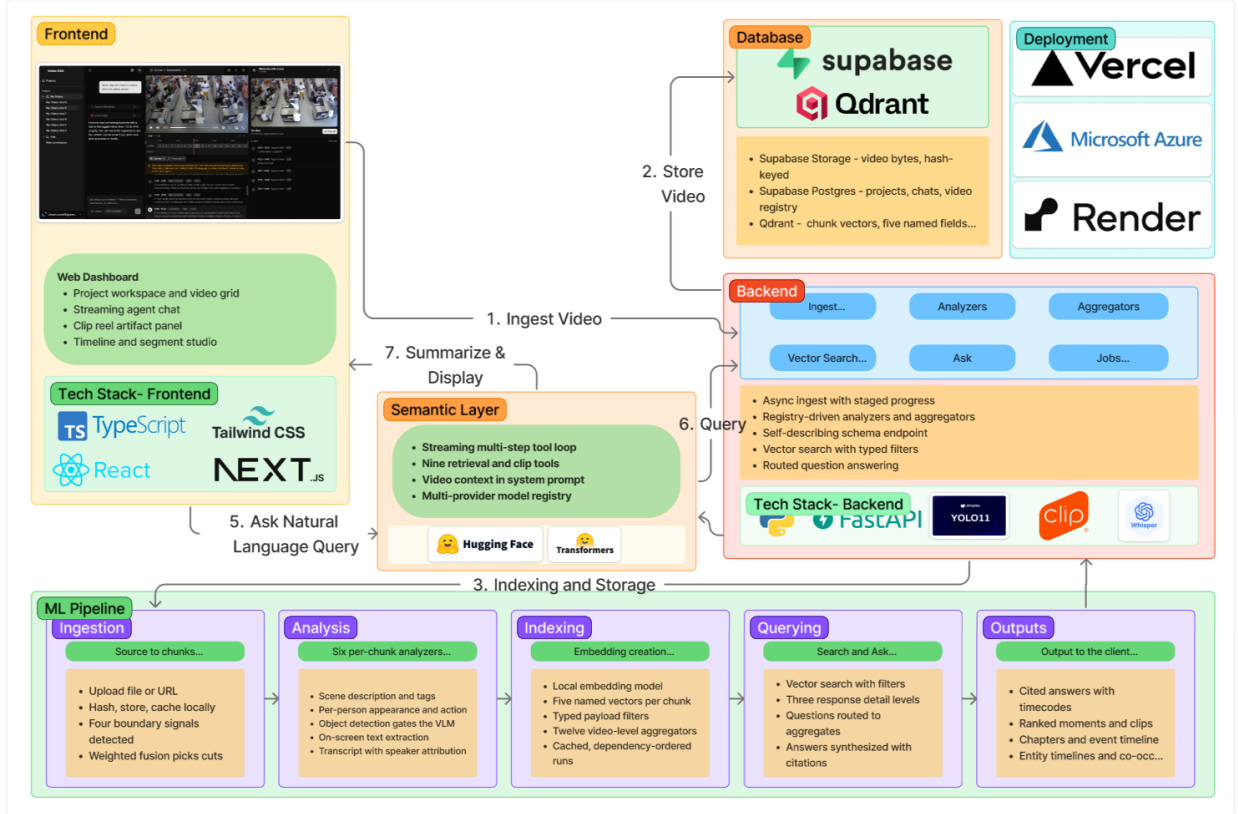


Figure 1: System architecture: client and agent tier, core analysis service, persistence, deployment targets, and the five-phase processing pipeline.

Composition inside the analysis service is deliberately in-process. Aggregators consume analyzer output directly rather than over HTTP, and HTTP is reserved for the external boundary. Analyzers and aggregators are held in registries, which is what makes NFR-8 achievable, because the ingest path, the store and the API iterate over a registry and never name a specific analyzer.

The analysis service has no concept of users, projects or row-level security. Any caller able to reach it can read every recording it holds. Multi-tenancy is therefore enforced entirely in the client tier, by row-level security on the database and by a single scoping function through which every agent tool resolves the recording identifiers it is permitted to pass downward.

Layer	Responsibility
Client	Marketing site and documentation, authentication, project workspace, video upload and status, agent chat, clip artifact panel, video studio timeline, per-recording detail view.
Agent layer	Multi-step streaming tool loop. Converts a natural-language request into search, question, inspection and clip operations, and streams results and artifacts back.
API route layer	The client’s own server routes: project scoping, row-level security, job reconciliation, and proxying to the analysis service.
Core service	Ingest orchestration, chunking, analyzers, aggregators, vector search, question answering, clip and poster export, job tracking.
Storage boundary	The single module that knows both a storage URL and a local path. Everything above it speaks URLs, everything below it receives paths.
Persistence	Object storage for video bytes, relational store for application state, document store for analysis output, vector store for chunk embeddings, local cache for decoded media and model weights.

Table 4: Layer responsibilities.

3.2 The Processing Pipeline

A recording passes through five stages. Most are parameterised rather than fixed, because surveillance footage varies too widely for a single configuration to suit every case.

1. **Chunk.** The recording is cut where its content changes. Four boundary detectors contribute evidence for a cut: a change of speaker, a period of silence, a hard scene cut, and a shift in the visual content of the frame. Their scores are fused under a weighting and the strongest peaks are taken as boundaries.
2. **Analyse.** Six analyzers are available and are selected at upload time, because their costs differ by orders of magnitude: scene description, per-person description, object detection, on-screen text recognition, transcription, and speaker-attributed transcription.
3. **Index.** Each chunk is stored as a single point carrying five separately embedded vectors and a typed, filterable payload. Embeddings are computed locally.
4. **Aggregate.** Twelve aggregators run over the saved analyzer output, in an order derived from their declared dependencies, producing the video-level material.
5. **Retrieve.** Searches return ranked moments with timecodes. Questions are routed to the video-level aggregates capable of answering them and synthesised with mandatory citations.

3.3 Ingestion Lifecycle

Ingestion is asynchronous. The service accepts a file or a link, returns immediately with a job identifier, and processes the recording on a background thread. The job reports its stage as fetching, chunking, analyzing, indexing, aggregating or complete, and the client polls it, reconciling the result into the recording’s row. Because jobs are held in memory, a restart mid-ingest is detected by the client, which either recovers the recording from its persisted records or fails the row explicitly so that re-indexing becomes the route forward.

3.4 Technology Stack

Four choices carry most of the design. Python was selected for the analysis service because every model the system depends upon is distributed as a Python library first. Qdrant was selected because storing several vectors per chunk under one point is a first-class capability there, which is what enhancement E-3 rests upon, and because it runs embedded so development requires no

Concern	Technology
Frontend	Next.js 16, React 19, TypeScript 5, Tailwind CSS v4, shadcn/ui on Radix primitives
Agent and models	Vercel AI SDK v6; OpenAI, Google, Groq, Cerebras and xAI providers behind one model registry
Client state	Zustand, TanStack Query
Playback	video.js, hls.js
Documentation	Fumadocs over MDX, served from the same application
Auth and tenancy	Supabase Auth with Postgres row-level security
Backend	Python 3.13, FastAPI, Uvicorn
ML runtime	PyTorch 2.11 (CUDA 13), Transformers
Boundary detection	pyannote.audio, Silero VAD, PySceneDetect, OpenCLIP ViT-B/32
Analysis models	Vision-language model for scene, people, object and text description; faster-whisper for transcription; YOLO11 and EasyOCR as detection gates
Aggregation models	GLiNER for zero-shot named-entity recognition, a transformer classifier for sentiment, an LLM for narrative aggregates
Embeddings	BGE-small-en-v1.5 via sentence-transformers, executed locally
Vector store	Qdrant, one point per chunk with five named vectors
Media I/O	OpenCV, PyAV, soundfile, Pillow
Application data	Supabase: Postgres, Auth and Storage
Deployment	Vercel (client), CUDA-capable host (core service), Supabase Cloud, Qdrant

Table 5: Technology stack.

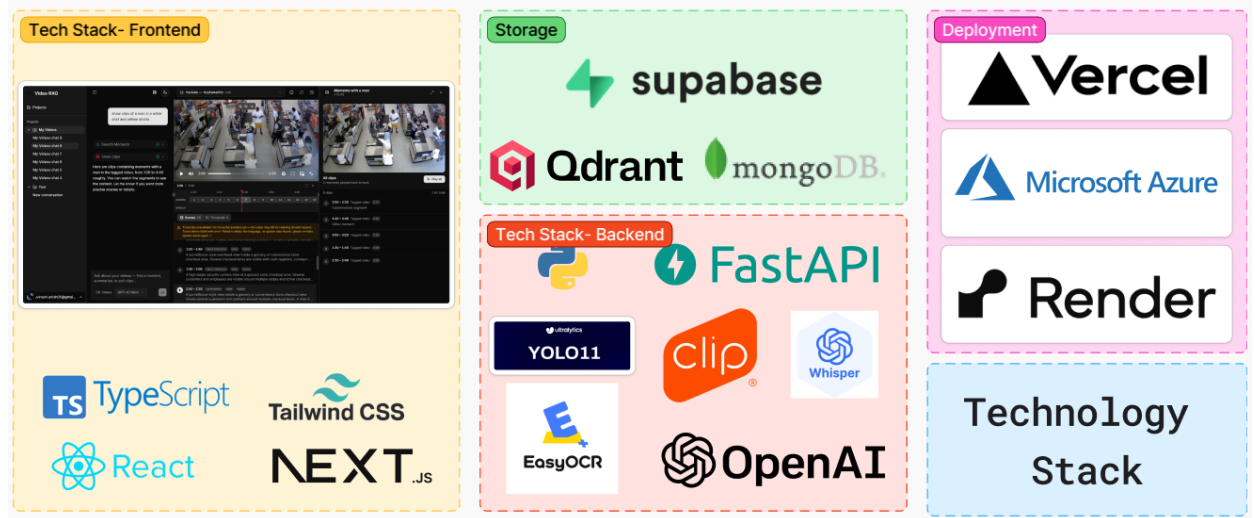


Figure 2: Technology stack across the frontend, backend, storage and deployment tiers.

separate server. A local embedding model was selected because embedding runs on every chunk of every recording, and paying an API per embedding would make re-indexing expensive. Supabase was selected because Postgres, authentication, row-level security and file storage from one service constitute the whole of the non-video state.

4 UI and Acceptance Planning

4.1 Interface Plan

The interface is planned around a single principle: the operator should never be required to scrub. Every screen terminates in a moment that plays.

Three planned decisions follow directly from the requirements. Progress is reported per recording rather than as a global spinner, so a workspace with several recordings processing at once stays

Screen	Operator activity
Landing and documentation	Presents the system and hosts the documentation: concepts, workspace guides, and the API reference.
Authentication	Registration and sign-in. Every subsequent route is guarded and every query is restricted to the signed-in account.
Projects	Lists projects, each grouping a set of recordings. Opening one enters its workspace.
Workspace	Adds recordings by file or link, selects analyzers and chunking mode, and observes each recording advance through Uploading, Indexing and Ready.
Agent chat	Accepts a description or a question and a selection of recordings. The assistant chooses the retrieval operations; answers stay concise and cite timecodes.
Clip reel panel	Opens beside the chat when the results are moments. A player above the list of moments; selecting one plays it, and they may be played consecutively.
Timeline studio	Presents scenes, transcript and chapters against a timeline that follows the playhead.
Recording detail	Five tabs over one recording: Overview, Chunks, Speech, People and objects, and the raw stored Record.

Table 6: Planned interface surfaces.

legible (FR-3). A recording not yet ready is excluded from search with a stated reason rather than silently returning nothing, because an empty result and an unfinished index are different situations that look identical to the operator. And every result carries its timecode as the primary handle, since the timecode is what turns a search result into footage the operator can watch.

4.2 Acceptance Planning

Every requirement is restated as a condition that can be observed to hold or not hold, together with the cases that establish it. A requirement with no case would be a requirement nobody agreed how to check, so the third column below is also the traceability matrix. The cases themselves are specified in Section 9.

Req.	Accepted when	Verified by
FR-1	An operator can register, sign in and group recordings into projects, and no account can reach another account’s data by any route.	AT-01 to AT-04, AT-06
FR-2	A recording can be added by file or link, the selected analyzers are the ones that run, and it reaches ready .	AT-07 to AT-09
FR-3	Stage and progress are visible throughout, the workspace stays usable, and a failure explains itself.	AT-11, AT-12, AT-17, AT-18
FR-4	A described event is returned within the leading results, and absent content is reported as absent.	AT-19, AT-20, AT-24
FR-5	Restricting by time, content or recording removes non-matching results and retains matching ones.	AT-21 to AT-23
FR-6	An answer is correct against the ground-truth sheet, carries a timecode for every claim, and is refused when unsupported.	AT-27 to AT-29
FR-7	Selecting a result plays the footage from that moment with no scrubbing, individually or as a reel.	AT-32, AT-33
FR-8	Summary, chapters, events, entities and anomalies are produced and legible without watching the footage.	AT-35, AT-37, AT-38
FR-9	A multi-turn conversation selects the right tools without the recording being named each turn, and survives reopening.	AT-30, AT-31
FR-10	The stored record for any chunk is retrievable, and the timeline follows the playhead.	AT-34, AT-36
FR-11	A recording can be re-examined at another depth without being re-added, and earlier results survive.	AT-14, AT-15
FR-12	Every retrieval capability is reachable over the documented API, and a client can be built from the capability endpoint alone.	AT-26, AT-39, AT-41
NFR-1	Search returns within 2s and an answer within 15s on a warm instance.	PT-1, PT-2
NFR-2	A recording becomes searchable within roughly three times its own duration.	PT-3
NFR-3	Stage and progress refresh at least every 5s during ingestion.	AT-11, PT-4
NFR-4	Playback of a selected moment begins within 2s, including later moments of a reel.	AT-32, AT-33, PT-5
NFR-5	No answer is produced without citations, and an unsupported question is declined.	AT-27, AT-29
NFR-6	Silent, cut-free, fixed-camera footage yields multiple usable chunks rather than one span.	AT-16
NFR-7	Deselecting an analyzer avoids its cost, a gated frame incurs no model call, and review incurs none.	AT-09, AT-26, AT-38, PT-3, PT-6
NFR-8	A new analyzer appears in the API and the ingest dialog after one module and one registry line.	AT-10, AT-40
NFR-9	Isolation holds at the database and the agent boundary, including against a fabricated identifier.	AT-03 to AT-06
NFR-10	The same file submitted twice is one recording, neither stored nor processed twice.	AT-13

Table 7: Acceptance criteria and traceability matrix.

5 Execution Plan

Development proceeds in eight phases. Each is validated end to end before the next begins, so that a defect in an earlier stage is not discovered through the behaviour of a later one.

II Phase 2: Detailed Design, Interface, Dependencies and Test Plan

Phase	Stage	Work
1	Chunking	Boundary detectors, weighted fusion, the three chunking modes and the duration bounds. Validated on cut-free footage before anything was built on it.
2	Analysis	Scene, people, object, text and speech analyzers; frame sampling; detector gating; and the registry that renders them interchangeable.
3	Indexing and retrieval	Local embedding, the five named vectors, the typed filter set, response detail levels and the search endpoint.
4	Aggregation	Video-level output, dependency ordering, caching, and person and object entity linking.
5	Question answering	Routing of questions to the aggregates capable of answering them, and answer synthesis with mandatory citations.
6	Application tier	Authentication and row-level security, projects and workspace, upload and status reconciliation, the agent tool layer, clip reel and timeline studio, and the detail view.
7	Documentation	The MDX documentation site covering concepts, workspace guides and the complete API reference, together with the public landing pages.
8	Hardening and reporting	Execution of the acceptance cases against a live server, the automated suites supporting them, error handling, deployment, and this document.

Table 8: Eight-phase execution schedule.

Member	Responsibility
Avinash Anish	Application tier and ingestion. The entire interface, covering authentication, projects and workspace, upload and status, agent chat, clip reel panel, timeline studio and the recording detail view, together with the agent tool layer and its project scoping. On the pipeline side, ingestion: source handling, object storage and caching, content-addressed identity, the four boundary detectors, weighted fusion and the three chunking modes, and the background job and progress reporting behind them.
Vaivaswat Verma	Analysis service and retrieval. Everything downstream of chunking: the analyzers with their frame sampling and detector gating, embedding and the vector store, filters and detail levels, the video-level aggregators including person and object entity linking, and question routing and answer synthesis. Owns the HTTP API through which these are exposed.
Shared	Architecture and design decisions, acceptance testing against a live server, deployment, and documentation.

Table 9: Distribution of work.

6 Detailed Design

6.1 Design Flow

The pipeline is a strict sequence, and each stage consumes only what its predecessor produced. Chunking receives a decoded video file and produces an ordered list of non-overlapping time spans plus a configuration key identifying the scheme that produced them. Analysis receives those spans and a video context supplying frames and audio on demand, and produces one structured record per chunk per analyzer. Indexing receives those records and produces one vector point per chunk per analyzer. Aggregation receives the saved records, not the video, and produces video-level output. Retrieval reads only the vector store and the aggregates. Nothing at query time reopens the video, which is the single decision the system’s cost profile rests on.

Stage 1: Chunking

Four detectors run once per recording, each returning a list of time and strength events with strength in $[0, 1]$.

Signal	Model or algorithm	Event derivation	Strength
Speaker change	pyannote/speaker-diarization-community-1, gated	The waveform is diarized into turns; a boundary is emitted at the midpoint between consecutive turns whose speaker label differs.	Constant 1.0
Silence	Silero VAD v6	A boundary at the midpoint of every gap between speech segments longer than 0.05 s, including the trailing gap.	$\min(1, g/2.0)$ for a gap of g seconds
Hard cut	PySceneDetect ContentDetector	A boundary at the start of every scene after the first.	Constant 1.0
Semantic drift	OpenCLIP ViT-B/32, frames at 1 fps	Consecutive sampled frames are embedded and compared; a boundary at each midpoint with raw strength $1 - \cos(e_i, e_{i+1})$.	Rescaled by the clip's own largest jump

Table 10: Boundary detectors.

Weights are renormalised over the signals that produced at least one event, so a preset expresses which signals matter rather than acting as a granularity dial. Each event then contributes a Gaussian bump to a score curve evaluated on a 0.1 s grid:

$$S(t) = \sum_{s \in S} \sum_{(t_i, a_i) \in E_s} w_s a_i \exp\left(-\frac{(t - t_i)^2}{2\sigma^2}\right), \quad \sigma = 1.0 \text{ s}$$

Boundaries agreed upon by several detectors reinforce one another. Peaks are selected greedily in descending order of $S(t)$, subject to a score floor of 0.12 and a minimum separation of 2.0 s. Three mutually exclusive modes are exposed: **preset** uses a named weighting, **weights** takes a caller-supplied one validated against the four signal names, and **interval** cuts fixed spans of N seconds with no detector run at all. For the weighted modes a span longer than $D_{\max}$ is divided into $\lceil d/D_{\max} \rceil$ equal parts, and a span shorter than the minimum is merged forward. The defaults are 5 s and 20 s. The output is the ordered spans plus a configuration key such as **audio-video:5-20**, which is carried on every downstream vector and is what permits two chunkings of one recording to coexist.

Stage 2: Analysis

The frame-selecting analyzers share a three-part sampling strategy. Candidate frames are drawn at a low fixed rate, near-duplicates are removed by comparing OpenCLIP embeddings against the last frame retained, and the survivors are thinned to a per-analyzer maximum. The scene analyzer derives its similarity threshold adaptively, as the 40th percentile of consecutive-frame similarity within the chunk, capped at 0.98. The cap is load-bearing: on a static chunk the percentile lands on the median, measured at 0.9955, and distinct frames are then mined out of compression noise. All frames selected for a chunk are sent in one request, so the model sees the chunk as a continuous sequence.

The vision-language model is invoked with strict structured output, so every required property is present and no additional properties are returned; downstream stages never parse prose. Two rules apply across the table. Whole-track passes are performed once and memoised, because a per-chunk diarization pass would label the same person **SPEAKER_00** in one chunk and **SPEAKER_01** in the next, and per-segment language detection produced fluent German from an English clip. And **transcript** and **diarization** are mutually exclusive, enforced by the API, because their embedded vectors were measured cosine-identical at 1.0000.

Analyzer	Models and gating	Structured output
<code>default_video</code>	Candidates at 2fps; OpenCLIP deduplication (adaptive threshold, cap 0.98); 4 to 8 frames retained; VLM at 768 px.	<code>description</code> , <code>setting</code> , <code>people[]</code> , <code>objects[]</code> , <code>actions[]</code> , <code>tags[]</code> , with the timestamps of the frames used.
<code>people</code>	YOLO11n gate at 1 fps, confidence 0.35; boxes below 900 px ² discarded; identifiers by overlap tracking; up to 6 frames with numbered yellow boxes; VLM at 1280 px.	Per person <code>box_id</code> , <code>appearance</code> , <code>clothing</code> , <code>role</code> , <code>action</code> , plus <code>locations</code> and a <code>people_count</code> .
<code>object_detection</code>	Shares the same YOLO11n pass; frames annotated with unlabelled cyan boxes; up to 6 frames; VLM at 1280 px.	<code>detections[]</code> of <code>object</code> , <code>description</code> and <code>context</code> , plus a filterable list of plain object names.
<code>ocr</code>	EasyOCR detection stage only; red boxes; similarity threshold 0.985 plus a layout-change test; VLM at 1600 px.	<code>texts[]</code> of verbatim <code>text</code> and <code>context</code> , plus a summary.
<code>transcript</code>	faster-whisper <code>tiny</code> ; language detected once over the whole track and pinned.	One plain-text transcription per chunk.
<code>diarization</code>	pyannote over the whole track once, memoised; faster-whisper segments per chunk; speaker by largest temporal overlap.	<code>turns[]</code> of <code>speaker</code> , <code>start</code> , <code>end</code> and <code>text</code> , plus a rendered form and the speakers present.

Table 11: Analyzers, their models and their outputs.

Stage 3: Indexing and Aggregation

Every analyzer implements `render_fields`, flattening its output into a map of named vector space to text. Five spaces are defined: `combined`, `description`, `people`, `actions` and `objects`, of which `combined` must always be present. A space is omitted rather than emitted empty, so a chunk in which the model saw nobody is not a candidate in a people-scoped search. Text is embedded locally with BAAI/bge-small-en-v1.5, producing normalised 384-dimensional vectors compared by cosine distance. The model family was trained with an instruction prefix on the query side only, so queries carry the prefix and documents do not; applying it to both, or neither, measurably degrades recall.

Twelve aggregators then run over the saved records in an order derived from their declared dependencies. Five incur API calls; the rest are free, which is what makes re-running them on

demand practical.

Aggregator	Depends on	Output
stats	None	Duration, chunk count, person count series with extremes and busiest and quietest spans, object frequencies, word count and speech ratio.
novelty	None	Cosine distance of each chunk from the recording centroid, ranked; outliers beyond two standard deviations, so unusual scales with how varied the recording is.
speaker_stats	diarization	Per speaker seconds, turns, words, share, longest turn and chunks, plus handover count and a turn timeline.
summary	None	A hierarchy built by repeated halving, depth derived from chunk count and bounded to three to six tiers, so leaf granularity does not vary with length.
chapters	None	Runs of consecutive chunks grouped into named sections. Chapters must be consecutive, non-overlapping and exhaustive.
events	None	Discrete chunk-anchored occurrences with actor and category, resolved to real time spans. Background activity is excluded.
ner	None	GLiNER zero-shot entities over descriptions, on-screen text and speech across ten labels, with pronouns excluded by a stop list.
sentiment	diarization	Per-turn timeline, per-speaker distribution and overall distribution. Speech only, reported as sentiment of language observed.
entities	people	Person observations linked into individuals by constrained clustering, then narrated.
object_entities	object_detection	Object sightings linked into individual objects, with fixtures counted but not tracked.
entity_timeline	entities	Per person first and last sighting, total span, observed presence summed over sighting spans, and appearance count.
cooccurrence	entities	Pairs of people sharing a chunk with counts, people per chunk, and the objects present.

Table 12: The twelve aggregators.

Entity linking deserves its own account, because it is where the naive approach fails hardest. Identity is decided by embedding and clustering, not by asking a model to match people. Measured on this footage, provably different people, meaning people co-visible within one chunk, reached 0.941 similarity, while known-same pairs sat between 0.889 and 0.915. Those bands overlap, so no threshold alone suffices and the constraints carry the weight. The signature is the **clothing** field alone, because blending in **appearance** dragged same-person scores to 0.69 to 0.87. Observations in one chunk above 0.95 are merged first, since the tracker loses a person who moves between sampled frames. Each observation’s mean similarity to all others is computed and anything at or above 0.80 is left unlinked, because it describes half the recording and identifies nobody. Pairs above 0.88 from different chunks are then merged greedily, rejecting any merge whose clusters share a chunk. Only then is an LLM used, and only for people seen more than once.

Stage 4: Retrieval and Question Answering

A search embeds the query with the query-side prefix, selects one of the five named vector spaces, applies the validated filter set, and returns ranked chunks with time spans and payloads. A score threshold is what makes no match expressible: cosine similarity always ranks something, so a query for absent content still returns its nearest neighbours. On this data, content actually present scored ≥ 0.665 and absent content ≤ 0.524 , so a threshold near 0.55 to 0.60 separates them.

A question takes a different path. Each aggregate carries a hint, a short description of what it is good for written in the vocabulary a question would use. The hints are embedded and compared against the question, and the aggregates standing at least one standard deviation above the mean score are selected, capped at three. If nothing stands out the single best match is used. Relative

selection is used because absolute thresholds do not survive contact with real questions: the score distribution shifts per question, with means between 0.47 and 0.56 measured here, so any single cutoff admits everything or nothing. The routed aggregates are rendered into labelled blocks carrying explicit timecodes, the overall summary is always included, and the synthesis prompt requires every claim to carry a `[video_id start-end]` citation and directs the model to state plainly when the material does not support an answer.

6.2 Database Design

Persistence is divided across four stores, because the kinds of state the system holds have incompatible access patterns and forcing them into one store would compromise all of them.

Store	Technology	Contents	Unit
Application database	Supabase Postgres	Accounts, projects, the recording registry, conversations and messages. Normalised, transactional, and subject to row-level security.	Row
Object storage	Supabase Storage, two buckets	Video bytes as uploaded, the canonical copy the pipeline reads, and one poster frame per recording.	Object
Analysis record store	JSON documents on local disk	Everything the analyzers and aggregators produced for one recording, nested under its chunks.	Document
Vector store	Qdrant, embedded	One point per chunk per analyzer, carrying five embeddings and a filterable payload.	Point

Table 13: The four persistence tiers.

Analysis output is held as documents rather than relations because its schema is defined by the analyzer registry, not by the database. Each analyzer declares its own output shape and a chunk carries one key per analyzer that ran on it. Modelling that relationally would make adding an analyzer a schema migration, which contradicts NFR-8. Application state is relational because it needs foreign keys, cascading deletes and per-row access control. Retrieval is separated again because approximate nearest-neighbour search over several named vector spaces is not something either of the other two can serve.

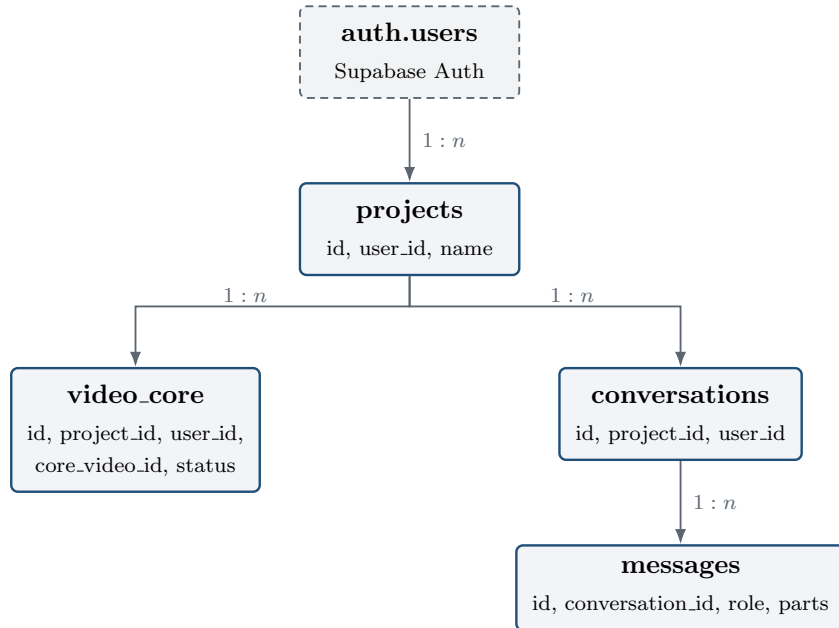


Figure 3: Application database. Every relationship cascades on delete, so removing a project removes its recordings, conversations and messages with it.

6.3 Normalisation

The relational half of the design is analysed below. Every relation uses a single surrogate UUID primary key, so no composite key exists anywhere and a partial dependency, which is what second normal form forbids, is structurally impossible. All four relations are therefore in at least 2NF by construction.

Relation	Form	Analysis
projects	BCNF	id determines <code>user_id</code> , <code>name</code> , <code>description</code> and the timestamps. No non-key attribute determines any other, and the only determinant is the key.
conversations	BCNF	Same structure. <code>project_id</code> is nullable, since a conversation need not be attached to a project, and the attachment is what scopes the agent's tools.
messages	BCNF	id determines <code>conversation_id</code> , <code>role</code> , <code>parts</code> and <code>created_at</code> . <code>parts</code> is <code>jsonb</code> , discussed below.
video_core	2NF, deliberately not 3NF	id determines every column, but <code>core_video_id</code> in turn determines <code>playback_url</code> , <code>poster_url</code> , <code>duration</code> and <code>size_bytes</code> , since all four are functions of the video bytes. This is a transitive dependency.

Table 14: Normal form of each relation.

Two departures from textbook normalisation are deliberate and are recorded here rather than glossed over.

- **The transitive dependency in video_core.** Decomposing into a `media(core_video_id, playback_url, poster_url, duration, size_bytes)` relation with `video_core` holding a foreign key would put the schema in 3NF. It was not done for two reasons. The duplication is bounded, since a row exists only per project that references the recording, and in practice a recording is referenced by one or two projects. And the dominant read is the workspace listing, which fetches a project's recordings with their posters and durations; under decomposition every listing pays a join for columns that are immutable once written. The dependency is also not fully determined at insert time, because `core_video_id` is null until the bytes have been downloaded and hashed, so the referenced row would not yet exist.
- **Non-atomic columns.** `analyzers` is a text array, `ingest_config` and `aggregates` are `jsonb`, and `messages.parts` is `jsonb`. Strictly this violates first normal form. The array is retained because both the interface and the agent evaluate the predicate does this recording have people analysis, which an array answers directly and a junction table answers with a join per row of a listing. The `jsonb` columns are retained because their shape is owned by another system: `ingest_config` is replayed verbatim on re-index rather than queried, and `parts` is a sequence of heterogeneous parts (text, tool calls, tool results and artifacts) whose shapes are set by the agent runtime, not by this schema.

The vector store payload is denormalised on purpose and is not a relational store at all. It duplicates fields already present in the record store so that ranking, filtering and citing a moment never require a join back to disk. The record store remains authoritative; the payload is a derived read model that is rebuilt whenever the recording is re-indexed.

6.4 Schema

Uniqueness in `video_core` is per project and never global, since `core_video_id` is a content hash and the same file added to two projects yields the same value. The unique index is declared over (`project_id`, `core_video_id`) and is partial, restricted to rows where the column is non-null. Two URLs are kept rather than one, because the system spans two Supabase projects: this one holds the row and the object the browser uploaded, while the analysis service's own project holds the canonical copy that plays. Indexes follow the reads: `video_core` on (`project_id`, `created_at`)

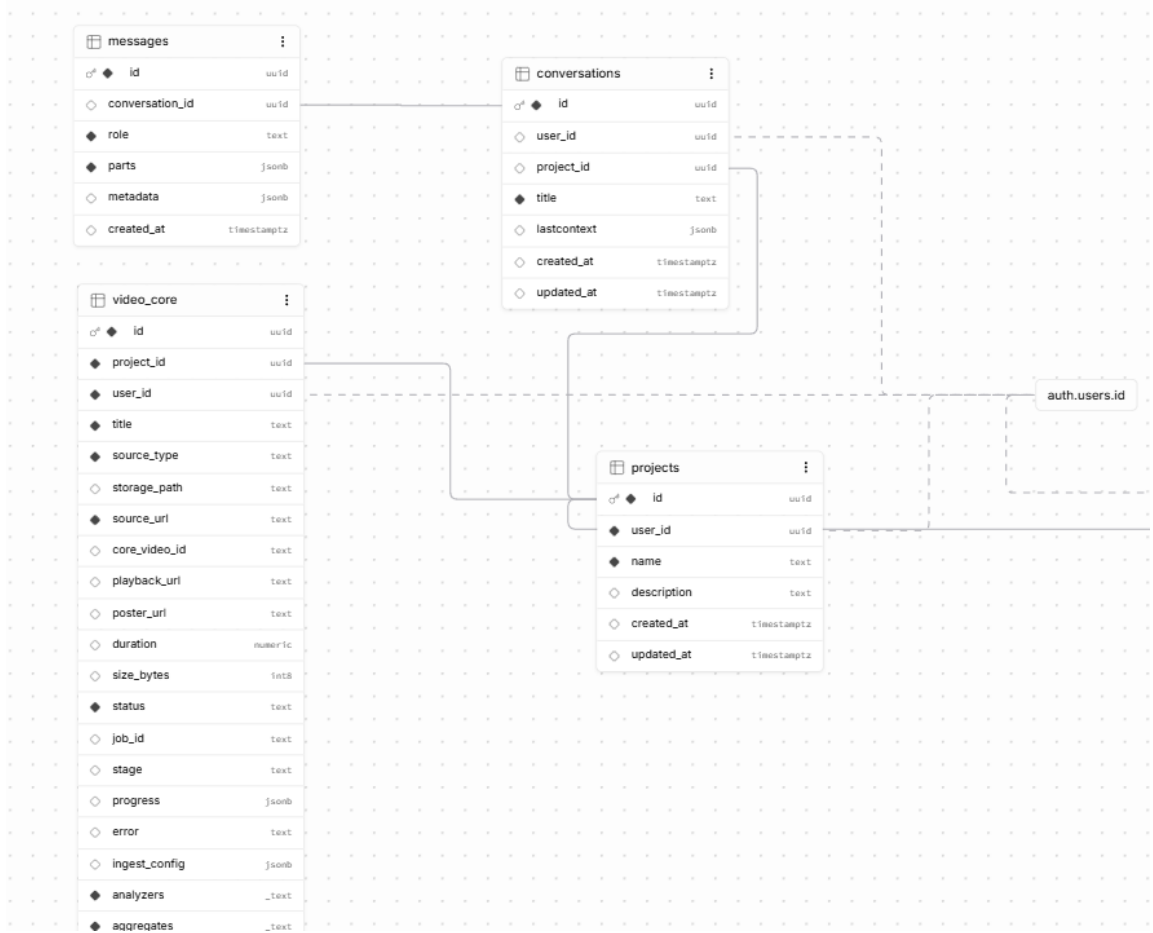


Figure 4: Application database as deployed: the four live relations with their columns, types and nullability, and their foreign keys into **auth.users**. Filled markers denote non-nullable columns.

Relation	Columns and notes
projects	id, user_id, name, description, timestamps. The grouping every other relation hangs from.
video_core	The application's view of a recording analysed by the core service, and the only place the two halves of the system meet. Identity and ownership (id, project_id, user_id, title); source and media (source_type, storage_path, source_url, core_video_id, playback_url, poster_url, duration, size_bytes); processing state (status, job_id, stage, progress, error); and what was asked for and produced (ingest_config, analyzers, aggregates, chunk_config, chunk_count).
conversations	id, user_id, project_id, title, lastContext, timestamps.
messages	id, conversation_id, role, parts, metadata, created_at.
videos	Superseded. Predates the current analysis service and is no longer read; video_core replaced it additively so no migration had to drop data.

Table 15: Application relations.

DESC) for the workspace listing and partially on **job_id** where non-null, which is the lookup reconciliation performs; **conversations** on **project_id**; and **messages** on (**conversation_id**, **created_at**), the order a conversation is replayed in. Every foreign key cascades on delete and a shared trigger function maintains **updated_at**.

The vector store holds a single Qdrant collection, **chunks**, with one point per recording, analyzer, chunk span and chunk configuration. Each point carries all five named vectors. The point identifier is a UUID derived deterministically from **sha1(video_id | analyzer_id | start | end | chunk.config)**, so re-ingesting the same chunk upserts in place. The key is built from the time

span rather than a positional index, because chunk 12 of one chunking run is a different moment from chunk 12 of another and a positional identifier would silently rebind a vector to the wrong timeframe.

Filter	Payload field	Match	Type
video_ids	video_id	any	list[str]
chunk_ids	chunk_id	any	list[int]
analyzer_ids	extractor_id	any	list[str]
chunk_config	chunk_config	exact	str
objects	objects	any	list[str]
tags	tags	any	list[str]
speakers	speakers	any	list[str]
people	people	any	list[str]
min_people	people_count	$\geq$	int
max_people	people_count	$\leq$	int
after	start	$\geq$	float
before	end	$\leq$	float

Table 16: The twelve declared filters. The same twelve payload fields are declared as indexes.

The filter set is declared in one place and the API passes a flat dictionary through to it, so a filter added to the declaration is reachable from the API immediately. An unrecognised filter name raises an error carrying the nearest valid name, because a filter that is silently ignored returns plausible but wrong results, which is worse than a failure. Deletion is scoped rather than wholesale: removing vectors takes an optional chunk configuration and analyzer identifier, because re-running one analyzer must delete only that analyzer’s vectors.

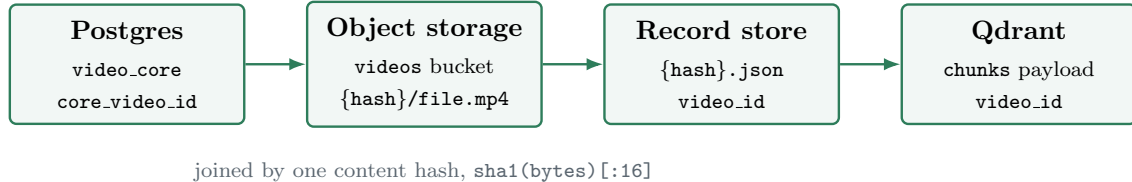


Figure 5: The content hash of the video bytes is the identity shared by every store, which is what makes ingestion idempotent (NFR-10).

6.5 Modularity

Modularity is enforced by three rules rather than by convention.

- **One module per concern.** In the analysis service, `chunking/` splits a recording, `analyzers/` describes each chunk, `aggregators/` derives video-level output, `vectordb/` embeds and indexes, and `api/` exposes it. In the client, routes sit under `app/`, agent tools under `app/api/agent/tools/`, the typed service client under `lib/core/`, and database access under `lib/supabase/`. No directory reaches past its neighbour’s interface into its internals.
- **Plug-in modules, registered rather than wired.** An analyzer is a single file exposing `id`, `analyze()` and `render_fields()`, plus one line in a registry. An aggregator likewise, declaring `depends_on` from which run order is derived. Ingestion, the stores and the API iterate the registry and never name a specific analyzer, so adding one edits one file. The client does not name them either, because the ingest dialog reads the service’s `/schema` endpoint.
- **Each boundary owned by exactly one module.** `storage.py` is the only place that knows both a storage URL and a local path; `paths.py` the only place that names a filesystem location; `api/core.py` holds the logic with `api/app.py` a thin HTTP layer over it; and `scope.ts` is the

one function every agent tool resolves recording identifiers through, so a new tool cannot forget tenancy.

Protocol base classes and dataclasses define what an analyzer or aggregator must provide, and TypeScript types in `lib/core/types.ts` mirror the service's response shapes, so a change on one side of the HTTP boundary fails to compile on the other rather than at runtime.

6.6 Scalability

The design decision that governs cost is that a recording is analysed once and queried many times. The expensive work, meaning the vision-language model, transcription and detection, runs at ingest and is written to disk as a permanent record. Every search and every question afterwards reads that record and its embeddings, so the marginal cost of a query is a local embedding and a vector lookup with no model call at all. Asking a hundred questions of a recording costs one analysis, not a hundred.

Five further properties keep that true as the corpus grows. Aggregators read records rather than video, so re-running one after tuning costs no re-analysis, and their results are cached and recomputed only on demand or when the analyzer set changes. Ingest is idempotent by content hash, so the same file submitted twice is one recording while two projects may still reference it independently. Vector deletion is scoped to the analyzer being re-run, so a recording can gain text recognition later while keeping every other analyzer's work, and vector keys carry the chunking configuration, so one recording can be indexed coarsely and finely at once. Cheap work gates expensive work, so a frame where nothing is detected and no text is read is never submitted to a model. And storage is layered by durability: records are authoritative, vectors are derived and rebuildable by local embedding with no model spend, and the media cache is regenerable, which is why no local path is ever persisted.

The stores scale along their own axes. Postgres carries only application state, so its row count grows with projects and conversations rather than with video duration. The vector store grows with chunks, at one point per chunk per analyzer; all twelve filterable payload fields are declared as Qdrant indexes, and although embedded mode ignores them and scans, pointing the same code at a Qdrant server gains real indexes with no code change. The record store grows with analysis depth and is the only store that holds anything expensive to recreate.

6.7 Security

Security rests on the fact that the analysis service has no concept of users. Every control is therefore placed where the identity actually exists.

Two residual exposures are recorded rather than claimed away. The storage buckets are public-read, so a playback URL is permanent and unsigned; the path begins with a content hash and is therefore unguessable, but it is not authenticated, and a deployment holding sensitive footage should move to signed URLs. And the analysis service trusts its caller entirely, so it must not be exposed to the public internet without the shared token set.

Control	Mechanism
Authentication	Supabase Auth. Every route beyond the public pages is guarded, and an unauthenticated request to a guarded route redirects rather than rendering.
Row-level security	Enabled on every relation. <code>projects</code> , <code>conversations</code> and <code>video_core</code> test <code>auth.uid() = user_id</code> directly. <code>messages</code> carries no owner column, so its policy tests ownership of the parent conversation through an <code>EXISTS</code> subquery. Enforcement is in the database, so it keeps holding as routes are added.
Agent boundary	Every agent tool resolves the recording identifiers it may touch through one scoping function bound to the caller’s project. A model that fabricates a video identifier, or repeats one from another conversation, has it filtered out rather than answered from another operator’s footage.
Service credentials	The service-role key is used only in server-side routes and never reaches the browser. <code>CORE_API_TOKEN</code> on the client must match <code>VIDEOMIND_API_TOKEN</code> on the service, so a deployed analysis service is not open to arbitrary callers.
Referential integrity	Every foreign key cascades on delete, so removing an account or a project leaves no orphaned recording, conversation or message.
Secrets	All provider keys and tokens are read from the environment. The gated diarization model additionally requires a Hugging Face token whose terms have been accepted.
Input handling	Filters are validated against a declaration and an unknown name is rejected. Server-side fetches of caller-supplied URLs are capped by <code>VIDEOMIND_MAX.BYTES</code> , default 4 GiB, so a link cannot be used to fill the disk.

Table 17: Security controls.

7 Detailed User Interface

The interface is a single-page application over eight surfaces, described below in the order an operator meets them.

Three interaction decisions distinguish it from a conventional media library. Progress is per recording, so a workspace processing several at once stays legible and usable throughout. A recording that is not yet ready is excluded from search with a stated reason rather than silently returning nothing. And the timecode is the primary handle on every result, which is what turns a hit into footage watched without scrubbing. The agent layer removes the query language from the operator’s job entirely: a request in plain words becomes the right retrieval operation, and context carries across turns without the recording being named again.

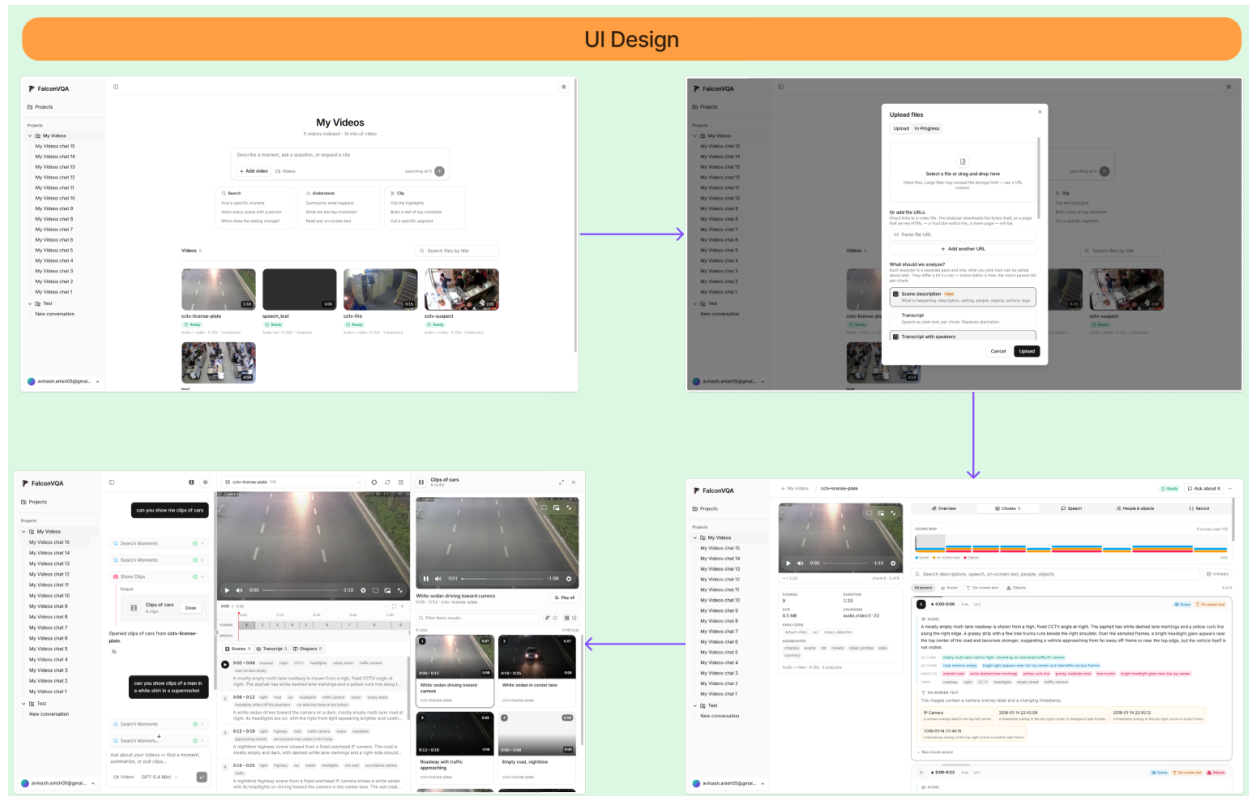


Figure 6: Operator interface. Clockwise from top left: the project workspace and video grid; the ingestion dialog, whose analyzer list is built from the analysis service’s own capability endpoint; the per-recording detail view with its chunk map and tabbed record; and the agent chat with the clip reel artifact panel and the timeline studio.

Surface	Detail
Landing and docs	Public pages plus the full MDX documentation site at <code>/docs</code> : getting started, concepts, workspace guides, architecture, troubleshooting and the API reference.
Authentication	Registration, sign-in and session middleware. Every subsequent query is restricted to the signed-in account by row-level security rather than by interface filtering.
Projects	A list of projects newest first, each grouping recordings and conversations. Creating one lands directly in its workspace.
Workspace	A grid of recording cards, each showing its poster frame, title, duration and status. Cards advance through Uploading, Indexing and Ready, with the current stage and a progress bar shown per card rather than as one global spinner.
Ingest dialog	Two tabs, file and link. Files are dropped or selected and go to object storage; links are handed to the service, which downloads them itself. The analyzer checklist and the chunking mode selector are built from the service’s capability endpoint, so they cannot go stale.
Agent chat	A prompt box with a model selector and a Videos button that tags which recordings to consider, shown as chips above the input. The agent runs a streaming multi-step tool loop over nine tools and answers concisely, citing <code>m:ss</code> timecodes. Conversations persist and reopen with their tool calls, results and artifacts intact.
Clip reel panel	Opens beside the chat whenever the results are moments. A player sits above the moment list; selecting a row plays that moment, and Play all plays them consecutively. Because a clip is a range rather than a file, the MP4 loads once and the panel seeks.
Timeline studio	Scene, transcript and chapter tracks against a timeline that follows the playhead, for reviewing what was indexed in the region being watched.
Recording detail	Five tabs over one recording: Overview for the video-level output, Chunks for the chunk map and per-chunk records, Speech for the transcript, People and objects for linked entities and their timelines, and Record for the raw stored document.

Table 18: Interface surfaces in detail.

8 Third-Party Libraries and Tools

8.1 Extent of the Dependency

The system is composed almost entirely from third-party components and trains nothing of its own. The analysis service declares 28 direct Python dependencies and the client approximately 100 direct npm packages. Every model in the pipeline is pre-trained and third-party: four boundary detectors, six analyzers and the models behind three aggregators. What is original is the composition, not the components. The chunking fusion and weight redistribution, the frame sampling strategy, the detector gating, the five-vector point schema, the constrained entity linking, the question routing and the whole application tier are written for this project.

A rough division by weight of what the system does: model inference, storage and the web runtime are approximately 100 percent third-party, while the pipeline logic, the retrieval schema, the agent tool layer and the interface are approximately 100 percent original code written against those libraries. Of the roughly 130 direct dependencies, every one is free and open source. The only recurring costs are metered services, listed in Section 8.3.

8.2 Principal Dependencies

Component	Licence	Cost	What depends on it, and the impact of losing it
PyTorch, torchvision, torchaudio	BSD-3	Free	The runtime every local model executes on. Irreplaceable without rewriting the analysis service.
transformers, sentence- transformers	Apache 2.0	Free	Model loading and the embedding interface. A swap is possible but would touch every model call site.
pyannote.audio	MIT (model gated)	Free	Speaker change detection and diarization. The model requires accepting terms and a Hugging Face token. Losing it disables one boundary signal and one analyzer; weight redistribution absorbs the signal loss.
Silero VAD	MIT	Free	Silence boundaries. Small and self-contained.
PySceneDetect	BSD-3	Free	Hard cut detection. Purely algorithmic, no weights.
open_clip_torch	MIT	Free	Semantic drift boundaries and frame deduplication. Load-bearing; without it, frame sampling has no similarity measure.
ultralytics (YOLO11)	AGPL-3.0	Free for open use	The detection gate for the people and object analyzers. The licence is the one genuine constraint in the stack: AGPL obligations apply to a networked deployment unless a commercial Ultralytics licence is purchased. A permissively licensed detector would substitute at the cost of retuning the gate thresholds.
easyocr	Apache 2.0	Free	Text region detection only; a vision model does the reading. Replaceable.
gliner	Apache 2.0	Free	Zero-shot named entity recognition. Losing it drops one aggregator, which the dependency system skips rather than fails.
qdrant-client	Apache 2.0	Free	The vector store, run embedded. Named vectors are the reason for the choice; a store without them would force the five-vector schema into five points.
OpenCV, PyAV, soundfile, Pillow	Apache 2.0 / BSD-3 / MIT	Free	All media decoding. PyAV bundles ffmpeg, so no system install is needed.
FastAPI, Uvicorn	MIT / BSD-3	Free	The HTTP surface and its generated documentation.
supabase-py, yt-dlp	MIT / Unlicense	Free	Object storage access and YouTube resolution. yt-dlp is the one dependency expected to need periodic bumping.
Next.js, React, TypeScript	MIT / Apache 2.0	Free	The entire client.
Vercel AI SDK	Apache 2.0	Free	The streaming tool loop and multi-provider model registry. This is what reduced the agent to defining nine tools rather than building a chat runtime.
shadcn/ui on Radix, Tailwind CSS	MIT	Free	Every interface primitive. shadcn components are vendored into the repository rather than installed, so they are ours to edit.
video.js, hls.js, mediabunny	Apache 2.0 / MIT	Free	Playback, seeking and the clip reel.
Fumadocs	MIT	Free	The MDX documentation site, served from the same application.
TanStack Query, Zustand	MIT	Free	Client caching, background revalidation and local state.

Table 19: Principal third-party components, their licences and their impact.

8.3 Paid and Metered Services

Service	Model	Use and exposure
Vision-language and LLM provider	Metered per token and per image	The only unavoidable running cost. Used for the four frame-reading analyzers, five of the twelve aggregators, and answer synthesis. Cost is controlled by detector gating, frame deduplication, per-upload analyzer selection and response detail levels. Everything else runs locally.
Supabase	Free tier, then paid	Postgres, Auth and Storage. The free tier is sufficient for the project; a production corpus would exceed the storage allowance first.
Vercel	Free tier, then paid	Client hosting. Optional; the client runs anywhere Node.js does.
Hugging Face	Free, token required	Model weight downloads and the gated diarization model.
Qdrant	Free (open source)	Run embedded, so no service is paid for. A hosted cluster is optional and would gain real payload indexes.
GPU host	Capital or hourly	A CUDA-capable machine for the analysis service. Developed on a local RTX 4060.

Table 20: Metered and paid services.

9 Acceptance Test Plan and Test Cases

9.1 Plan

Forty-one acceptance cases and six measured non-functional cases run against a deployed instance over HTTP and through the browser, not against modules in isolation, because several defects found during development, including stale background jobs, broken answer synthesis and media paths that resolved locally but not remotely, were visible only with the whole system running. Cases are grouped by the area they exercise and run in order within a group, since later cases often depend on state earlier ones create. Search and question cases are scored against a written ground-truth sheet rather than a tester’s impression, so a re-run on a later build is comparable as a number.

Severity is assigned to a failure as follows. Critical means the system cannot be used for its purpose, or a tenant can reach another tenant’s data. Major means a stated requirement is not met but a workaround exists. Minor means the result is correct but the presentation or the timing is not what was intended.

Element	Configuration under test
Analysis service	Python 3.13 and FastAPI on a CUDA-capable host, RTX 4060 with 8 GB VRAM, 16 GB system RAM. Served over HTTP with Uvicorn, not the reload server.
Client	Next.js production build, deployed on Vercel, pointed at the analysis service by environment variable.
Application data	A dedicated Supabase project with row-level security enabled and the migrations applied from a clean state.
Vector store	Qdrant in embedded mode, backed by a data directory emptied before a full pass.
Browsers	Chrome and Firefox, current stable, desktop viewport.
Accounts	Two registered accounts, A and B, used for every isolation test. Neither is an administrator.
Model providers	Live API keys and a Hugging Face token. Tests are not run against mocked providers, because provider behaviour is part of what is being accepted.

Table 21: Test environment.

Latency cases are run warm, meaning model weights are resident and the media cache holds the recording. Cold-start timings are recorded separately rather than counted against the response

targets. Entry to a test pass requires the pipeline to run end to end on at least one fixture, the client deployed and reaching the service, migrations applied, both accounts created and ground-truth sheets written. Exit requires every Critical and Major case to pass with no such defect open, every open Minor defect recorded with the reason it is accepted, and every requirement traced to at least one passing case. Testing suspends if the service is unreachable, a provider is erroring, or a defect prevents ingestion from completing, and resumes from the start of the group in which it was suspended rather than the individual case, because a suspended run may have left partial state.

9.2 Test Cases

ID	Input	Expected output
AT-01	Registration and sign-in. A previously unused address and password; sign out; sign in again; then request a guarded route while signed out.	Account created and session established; re-sign-in succeeds. The guarded request redirects to sign-in rather than rendering the route or returning data.
AT-02	Projects. Two projects created as A; the list reopened; one opened; that one renamed.	Both listed newest first; opening enters its workspace; the new title appears in the list without a reload.
AT-03	Cross-account isolation. A owns a project with a ready recording and a saved conversation. Sign in as B and inspect all three lists.	B's project, recording and conversation lists are empty. No row belonging to A appears in any of them.
AT-04	Access by identifier. A's project, recording and conversation UUIDs requested as B, first by direct URL and then against the client's own API routes.	Every request refused or empty. Enforcement is row-level, so no route returns A's rows. This is not interface filtering.
AT-05	Fabricated identifier. As B, a chat message naming a recording identifier that belongs to A.	The scoping function drops the identifier. The agent answers from B's own recordings or states the recording is not in this project.
AT-06	Cascading delete. Delete A's project holding recordings, a conversation and messages, then query the child tables directly.	The recording, conversation and message rows are gone. No orphaned row remains in any table.

Table 22: Accounts and tenancy, AT-01 to AT-06.

ID	Input	Expected output
AT-07	Upload. <code>cctv-shopping.mp4</code> (5 min, 48MB) through the ingest dialog, default analyzers, preset <code>audio-video:5-20</code> .	Card appears at once with a poster frame; status runs pending to uploading to queued to analyzing to ready ; <code>chunk.count</code> and <code>duration</code> are populated.
AT-08	Link. A reachable direct MP4 URL pasted into the link tab.	The service fetches and hashes the file; the row reaches ready exactly as an upload does; <code>source_url</code> and <code>playback_url</code> both resolve.
AT-09	Analyzer selection. Ingest with <code>people</code> and <code>ocr</code> deselected, leaving <code>scene</code> and <code>transcript</code> .	The produced list is exactly the two selected; no chunk record carries a deselected analyzer’s key, and no cost was incurred for either.
AT-10	Discovered analyzer list. The ingest dialog’s list compared against the response of <code>GET /schema</code> .	The two sets agree exactly, because the dialog builds its list from the endpoint.
AT-11	Progress and concurrency. A 5-minute recording ingesting; the card watched while another recording is opened and searched.	stage advances monotonically through fetching, chunking, analyzing, indexing and aggregating, refreshing at least every 5s; the concurrent search returns normally.
AT-12	Unready recording. A mid-ingest recording tagged into a search or a question.	The interface states that the recording is still being processed. It does not return an empty result set.
AT-13	Idempotence. <code>cctv-shopping</code> already in project P; the identical file re-added to P, then added to a second project Q.	P recognises it by <code>sha1(bytes)[:16]</code> : no upload, no re-analysis, no duplicate row. Q gets its own row, but both share one <code>core_video_id</code> , one stored object and one record document.
AT-14	Second chunking depth. A recording indexed at <code>audio-video:5-20</code> , re-indexed at <code>interval:10</code> ; then a search scoped to each.	Both chunkings coexist; the first run’s vectors are not overwritten; each scoped search returns only that configuration’s chunks.
AT-15	Added analyzer. A recording indexed without <code>ocr</code> , re-run with <code>ocr</code> added.	OCR output appears, and every previously produced analyzer’s records and vectors are intact.
AT-16	Silent, cut-free footage. <code>cctv-shopping</code> , no speech and no hard cuts, ingested with the default preset.	Multiple chunks with differing descriptions, not one span. Weight redistribution puts the load on the signals that fired.
AT-17	Bad source. A URL that does not resolve to a video.	The row reaches failed , the reason is retained and shown on the card, and the workspace is not left loading.
AT-18	Lost job. The service restarted mid-ingest while the client polls <code>GET /jobs/{id}</code> .	The client detects the job is gone and either recovers from the persisted record or fails the row so re-indexing is offered. Polling stops.

Table 23: Ingestion and processing, AT-07 to AT-18.

ID	Input	Expected output
AT-19	Description search. Five events from the <code>cctv-shopping</code> sheet, queried in wording that does not reuse the stored description.	For each query the correct moment ranks within the top three, and its span is within one chunk of the sheet’s timelinecode.
AT-20	Vector space scoping. One person-by-clothing query issued against the <code>people</code> vector, then against <code>combined</code> .	The people-scoped search ranks the correct person higher, having matched a short person description rather than the whole record.
AT-21	Time filter. An event known to occur late, queried unrestricted, then with <code>before=60</code> .	Unrestricted returns the moment; restricted excludes it, and every returned span lies inside the window.
AT-22	Content filters. <code>objects=[known object]</code> ; then <code>min_people=3</code> ; then <code>speakers=[a speaker]</code> on <code>chernobyl-audio-video</code> .	Each removes non-matching chunks and retains matching ones. <code>min_people</code> answers a counting query that description similarity alone gets wrong.
AT-23	Recording scope. The same query with <code>video_ids</code> naming both ready recordings, then only one.	The scoped run returns results from the selected recording only.
AT-24	Absent content. A query for something the sheet confirms is not in the footage.	The score threshold suppresses the nearest neighbours and the result is empty, rather than an unrelated chunk being presented as a match.
AT-25	Unknown filter. <code>POST /query</code> carrying a misspelled filter name.	The request fails with an error naming the nearest valid filter. It is not served with the filter silently ignored.
AT-26	Detail levels. One identical five-hit query at <code>minimal</code> , <code>standard</code> and <code>full</code> .	All three return the same ranked moments; the payload grows with the level, and <code>minimal</code> stays economical for an agent.

Table 24: Search and filtering, AT-19 to AT-26.

ID	Input	Expected output
AT-27	Cited answers. Five questions from the cctv-shopping sheet, each answer checked against it and each citation followed.	Every answer is correct against the sheet; every claim carries a timecode; following a citation plays footage that supports the claim.
AT-28	Routing. Three questions, about a specific person, about what was unusual and about how busy the store was, with routing recorded.	They route to entities, novelty and statistics respectively, and routing is reported so the answer can be traced.
AT-29	Unanswerable question. A question about something the footage does not show, phrased as though it does.	The system states that the footage does not support an answer. It neither synthesises a plausible one nor cites an unrelated moment.
AT-30	Multi-turn context. An opening question, then three follow-ups referring back by pronoun without naming the recording.	Context carries across turns; the agent selects search, question or inspection operations appropriately.
AT-31	Persistence. Leave the page, sign out, sign back in, reopen the conversation.	Every turn is restored in order, including tool calls, results and artifacts, and continuing retains the earlier context.
AT-32	Play a result. One moment selected from a search result list.	Playback begins at that moment’s start timecode with no manual scrubbing, and stops at its end.
AT-33	Clip reel. Several moments from one recording, played with Play all.	They play one after another; the MP4 is loaded once and seeked, so later moments start without a fresh load.
AT-34	Timeline studio. A ready recording played, its tracks watched, then the playhead scrubbed elsewhere.	The timeline follows the playhead, the region shown corresponds to what was indexed there, and scrubbing moves the tracks with it.
AT-35	Video-level review. The detail view for cctv-shopping with aggregates computed, read without watching and compared to the sheet.	Summary, chapters, events, entities and anomalies are produced and legible; anomalies flag the moments the sheet marks unusual.
AT-36	Index inspection. A chunk selected from the chunk map and its record read across the tabs.	The description, on-screen text, people, objects and speech for that chunk are all retrievable, including the raw document.
AT-37	Entity linking. A fixture in which one person appears several times; one person’s timeline followed.	Separate sightings link into one individual with an ordered account of what they did. Two people visible in the same chunk are never merged.
AT-38	Aggregate caching. The detail view reopened and aggregates re-requested; then the analyzer set changed.	Cached results return with no re-analysis and no model spend. The change marks the cache stale and triggers recomputation.
AT-39	Capability-only client. The response of <code>GET /schema</code> and nothing else used to construct and issue a filtered search.	The endpoint describes every analyzer, vector field, filter and aggregator needed, and the request succeeds.
AT-40	New analyzer. A trivial analyzer registered as one module and one registry line; the service restarted.	It appears in the dialog and in <code>/schema</code> , with no edit to ingestion, storage, the API or the client.
AT-41	Documented API. Each endpoint of the API reference issued against the running service.	Every documented endpoint exists, accepts the documented parameters and returns the documented shape.

Table 25: Question answering, review, playback and programmatic access, AT-27 to AT-41.

ID	Measures	Input	Expected output
PT-1	Search latency	Ten distinct queries against a ready 5-minute recording on a warm instance, each repeated ten times.	Median and slowest wall-clock to ranked moments at the API and to rendered results in the browser. Both ≤ 2 s (NFR-1).
PT-2	Answer latency	Ten questions spanning the routed aggregates, same conditions.	Median and slowest time to a cited answer, ≤ 15 s (NFR-1).
PT-3	Processing time	cctv-shopping submitted with the default analyzer set against a cold record store.	Wall-clock from submission to ready over the recording’s duration, $\leq 3 \times$ (NFR-2), with the per-stage breakdown from bench.py and a count of model calls.
PT-4	Progress refresh	One full ingest observed on the recording card from submission to ready .	Interval between successive stage or progress updates, ≤ 5 s throughout (NFR-3).
PT-5	Playback latency	A first moment and a later moment selected within one clip reel.	Time from selection to the first frame at that timecode, ≤ 2 s for both, the second without a fresh MP4 load (NFR-4).
PT-6	Response size	One identical five-hit query at each of the three detail levels.	Token count per level, recorded rather than bounded, so the cost of the agent path is known rather than assumed.

Table 26: Non-functional cases. Each is measured warm, repeated ten times, and reported as the median with the slowest of the ten, since a target met on average but missed regularly is not met.

9.3 Test Automation

A case is worth automating when it runs repeatedly and a machine can check its pass condition: a status, a rank, a count, an identifier. One whose pass condition is a judgement about footage stays manual, since a machine scoring it would only be checking one model against another.

Layer and tooling	Coverage	Cases
Unit and contract tests, <code>pytest</code> , in <code>core/tests/</code>	The analysis service without a GPU or a provider: the analyzer and aggregator registries and their dependency ordering, chunking and weight redistribution on synthetic events, the vector key and chunking configuration, the filter declaration, scoped search and scoped deletion against an embedded Qdrant, the three detail levels, aggregate caching and staleness, background job success and failure, and the FastAPI surface through <code>TestClient</code> .	AT-09, AT-10, AT-13 to AT-17, AT-22 to AT-26, AT-36, AT-38 to AT-41
API regression, Postman collection run headless via Newman	Every endpoint in the API reference against a running service, asserting status, response shape and the capability endpoint's contents.	AT-25, AT-26, AT-39, AT-41
Retrieval scoring, a Python harness over the ground-truth sheets	Issues recorded queries against a live server and asserts the rank and resolved timecode of the expected moment.	AT-19 to AT-24
Pipeline benchmarks, <code>scripts/bench.py</code>	Per-stage timings for detectors, fusion, chunking and export, extended to record search and answer latency.	PT-1, PT-2, PT-3
Tenancy checks, SQL assertions run as each test account	Reads the other account's rows in every table and asserts nothing is returned.	AT-03, AT-04, AT-06
Browser flows, Playwright	Sign-in, project creation, ingest to <code>ready</code> , search, and playback beginning at the expected timecode.	AT-01, AT-02, AT-07, AT-11, AT-32
Manual, a structured checklist against the ground-truth sheets	Whether a description fairly accounts for the footage, a summary reads sensibly, linked people are the same person, and an answer is genuinely supported by the moment it cites.	AT-27, AT-28, AT-35, AT-37

Table 27: Automation layers and their coverage.

The remaining cases run from the case scripts against the deployed instance, with the automated layers as the regression net around them. Where a case appears in more than one layer the two halves differ: the `pytest` suite fixes the mechanism, for instance that a scoped delete removes only one analyzer's vectors, while the live-server case fixes the outcome on real footage. Two constraints bound how far automation goes. A full pipeline run needs a GPU and live model providers, so no hosted runner will serve it, and a full ingest of the principal fixture costs real money each time. The suite is therefore split by what it needs: `pytest` needs neither, since it stubs the embedder and drives an embedded Qdrant, so it runs on every commit anywhere; API regression, tenancy checks and browser flows run against a pre-indexed fixture on every change; and retrieval scoring and benchmarks, which require a fresh ingest, run before a release.

Each executed case is recorded with its identifier, the build and fixture it ran against, the observed result, and a verdict. A failure is logged with its severity, the interaction that produced it, and whatever the system reported at the time. Because ingestion runs as a background job, a failure during processing is recorded together with the stage the job had reached and the reason retained on the recording row, since that is usually the only account of what went wrong. A defect is closed when the case that found it passes on a later build and the cases around it in the same group still pass.

10 Documentation of Process, Design, ATP and ATCs

Documentation is treated as four artefacts with four different audiences, each kept where it is used rather than collected into one place that nobody opens.

Artefact	Location	Covers
Process	latex/ execution plan	The eight phases and their validation gates, the distribution of work, and a running record of decisions that look odd but are deliberate, each with the measurement that produced it.
Design	latex/	The software design document: problem, solution, requirements, enhancements, pipeline design, architecture, database and interface design. This report consolidates it across the four phases.
Acceptance test plan	latex/, Section 9	Environment, fixtures, entry, exit and suspension criteria, the automation split, and defect recording.
Acceptance test cases	latex/ Section 9, core/docs/ FalconVQA. postman-collection.json	Forty-one functional and six non-functional cases, each with its input and its pass condition. The API cases exist additionally as executable Postman requests.
Developer and user	frontend/content/docs/at/docs, plus one README per tier	Getting started, configuration, concepts (pipeline, chunking, analyzers, aggregators, retrieval, glossary), workspace guides, architecture, troubleshooting, and the full API reference.

Table 28: Documentation artefacts.

Two properties keep the documentation from drifting away from the system. The API reference is executable: every endpoint it describes exists as a Postman request run against the service, so one that no longer behaves as documented fails a test rather than being quietly wrong (AT-41). And the `/schema` endpoint describes the analyzers, filters and aggregators of the running instance, so a client can be built without reading the source, which AT-39 verifies by doing exactly that. Ground-truth sheets sit beside their fixtures, and the defect record sits alongside each test pass.

III Phase 3: Implementation

11 Coding Standards

- **Two independent tiers.** `core/` is the Python 3.13 and FastAPI analysis service; `frontend/` is the Next.js and TypeScript client. Each runs, builds and pins its dependencies on its own, and they meet only over HTTP. `datasets/` holds the test recordings and `latex/` the design documents.
- **One module per concern.** The directory layout is the architecture, as set out in Section 6.5. No directory reaches past its neighbour's interface into its internals, and the one place two concerns are legitimately valid at once, a storage URL and a local path, is a single module by design.
- **Plug-in modules, registered rather than wired.** An analyzer is a single file exposing `id`, `analyze()` and `render_fields()` plus one registry line; an aggregator likewise with `depends_on`. Adding one edits one file, and the client picks it up without a change because the ingest dialog reads `/schema`.
- **Typed interfaces between modules.** Protocol base classes and dataclasses define what an analyzer or aggregator must provide, and TypeScript types mirror the service's response shapes, so a change on one side of the HTTP boundary fails to compile on the other rather than at runtime. Filters are validated against a declaration and structured model output is requested under a strict schema, so no downstream stage parses prose.

- **Naming and layout follow each language’s conventions.** Python modules are lower-case with underscores, functions are verbs, and registries are the plural of what they hold. TypeScript components are Pascal case in their own files, hooks are prefixed `use`, and route handlers sit at the path they serve. Environment variables share one prefix per tier and every path is overridable by one.
- **Agile, phase by phase.** Development ran in eight iterative phases, each validated end to end against a live server before the next began, so a defect in an early stage was never diagnosed through the behaviour of a later one. Verification was performed against a running server rather than by code inspection, because several defects were observable only over HTTP.
- **Measurement precedes commitment, and the measurement is recorded.** Every threshold and sampling choice was established by executing it against real footage and comparing outcomes. Where the obvious choice was adopted first and observed to fail, the reasoning sits in a comment beside the code and in a decisions file, so the next person does not undo it.

12 User Implementation

The interface is implemented as described in Section 7 and shown in Figure 6. All eight surfaces are built and reachable: landing and documentation, authentication, projects, workspace, ingest dialog, agent chat with its clip reel panel, timeline studio, and the five-tab recording detail view.

Three implementation decisions make it smart rather than merely complete. Progress is per recording rather than a global spinner, so a workspace processing several at once stays legible and usable throughout, and the grid polls only the recordings that are actually in flight (AT-11). A recording not yet ready is excluded from search with a stated reason instead of silently returning nothing, because an empty result and an unfinished index otherwise look identical (AT-12). And the timecode is the primary handle on every result, which turns a hit into footage watched without scrubbing (AT-32, AT-33).

The agent layer removes the query language from the operator’s job. A request in plain words becomes the right retrieval operation over a streaming tool loop across nine tools; context carries across turns without the recording being named again; and conversations, including tool calls, results and artifacts, persist and reopen intact (AT-30, AT-31). The analyzer checklist and chunking selector are built from the service’s capability endpoint rather than from a list compiled into the client, so a new analyzer appears in the interface with no client change at all (AT-10, AT-40).

13 Scalable and Sustainable Design

The scalability argument is given in full in Section 6.7. Its shape in the implementation is that a recording is analysed once and queried many times: the expensive work runs at ingest and is written to disk as a permanent record, and every search and question afterwards reads that record and its embeddings. The marginal cost of a query is a local embedding and a vector lookup with no model call at all.

The database scales the same way by design. Analysis output is stored as documents because its schema is defined by the analyzer registry, and modelling it relationally would make adding an analyzer a schema migration. Application state stays relational, where foreign keys, cascading deletes and row-level security belong, and that security lives in the database rather than the interface, so it keeps holding as routes and tools are added. All twelve filterable payload fields are declared as Qdrant indexes; embedded mode ignores them and scans, which is correct at this scale, and pointing the same code at a Qdrant server gains real indexes with no code change.

The test design follows the same shape. Cases are grouped by area and ordered within it, so a group re-runs alone after a change. Every case names its fixture. Search and question cases are scored against a written ground-truth sheet rather than a tester’s impression, so a re-run on a later build is comparable as a number. And every requirement traces to at least one case, so a new requirement is visibly untested until a case exists for it.

Sustainability rests on the same registry that provides modularity. A new analyzer, a new

aggregator or a new filter is one module and one declaration, and nothing in ingestion, storage, the API or the client has to learn about it. Storage is layered by durability, so losing the cheap layers costs compute and never data: records are authoritative, vectors are derived and rebuildable by local embedding with no model spend, and the media cache is regenerable, which is why no local path is ever persisted anywhere.

14 Dataset Creation

No public benchmark matches the target case, which is fixed-camera surveillance footage with no cuts and often no speech. Seven recordings totalling approximately 125 MB were assembled from publicly available footage into **datasets/**, each isolating a property the others do not.

Fixture	Content	Property it isolates
cctv-shopping	5 min, 1280×720, fixed camera, supermarket, no speech, 48 MB	The principal fixture: neither cuts nor audio, the case general-purpose tools fail on. Carries the chunking, scene, people, object and text cases.
chernobyl-audio-video	60 s, several speakers, 7 MB	The only fixture where speaker change, silence and speaker-attributed transcription all fire. Carries every audio path.
cctv-license-plate	Short fixed-camera clip, vehicle plates, 9 MB	Text recognition against low-contrast, small-format on-screen text.
cctv-fire	Short fixed-camera clip, indoor fire, 19 MB	Novelty detection where the anomalous moment is known in advance.
cctv-car-fire	Short fixed-camera clip, vehicle fire, 14 MB	Event extraction on a second, visually unlike anomaly, so novelty is not tuned to one clip.
cctv-suspect	Short fixed-camera clip, suspected theft, 18 MB	Person description, and the moments used for clip reel testing.
cctv-accident	Short fixed-camera clip, road accident, 9 MB	A third anomaly class, and multi-object scenes for object entity linking.
Extended set (gap)	Longer footage, multiple cameras, one shared subject	Needed for cross-recording person linking; not yet assembled, which is why that feature is deferred rather than built untested.

Table 29: Test fixtures and what each one isolates.

Completeness is judged by pipeline coverage rather than by volume. Between them the fixtures exercise all four boundary detectors, all six analyzers, all twelve aggregators, every filter type, and both the answerable and unanswerable question paths. For each, a ground-truth sheet is written before testing begins, recording what occurs and at which timecode. Search and question cases are scored against that sheet, which is what makes AT-19 to AT-24, AT-27 and AT-35 reproducible numbers rather than opinions. The one gap, footage sharing a subject across cameras, is recorded above rather than papered over, and it is the reason cross-recording entity linking is listed as future work rather than shipped untested.

IV Phase 4: Product, Performance and Delivery

15 Product Demo: Completeness

The demonstration is given live. This section lists what will be shown and the requirement each item discharges, so the demonstration can be followed against the specification rather than watched as a tour.

#	Shown	Demonstrates	Covers
1	Register, sign in, create a project	Account creation, guarded routes, project listing and workspace entry.	FR-1
2	Sign in as a second account	The second account sees none of the first account’s projects, recordings or conversations, enforced in the database.	FR-1, NFR-9
3	Upload a recording and paste a link	Both ingestion paths, the analyzer checklist built from the capability endpoint, and the chunking mode selector.	FR-2
4	Watch the card advance	Per-recording stage and progress through fetching, chunking, analyzing, indexing and aggregating, with the rest of the workspace still usable.	FR-3, NFR-3
5	Re-add the same file	Recognised by content hash: no upload, no re-analysis, no duplicate row.	NFR-10
6	Search by description	Ranked moments with timecodes for an event described in words that do not repeat the stored description.	FR-4
7	Search with filters	Time window, object, minimum person count, speaker, and recording scope, each narrowing the result set.	FR-5
8	Search for absent content	The score threshold returns nothing rather than presenting the nearest unrelated chunk as a match.	FR-4, NFR-5
9	Ask a question	A synthesised answer with a timecode citation on every claim, and the routed aggregates reported alongside.	FR-6, NFR-5
10	Ask an unanswerable question	The system states the footage does not support an answer.	NFR-5
11	Play a result, then a reel	Playback beginning at the moment with no scrubbing, and several moments played consecutively from a single media load.	FR-7, NFR-4
12	Open the recording detail view	Summary, chapters, events, linked people and objects with their timelines, and anomalies, read without watching the footage.	FR-8
13	Open the chunk map and timeline studio	The stored record for an individual chunk, and scene, transcript and chapter tracks following the playhead.	FR-10
14	Multi-turn conversation, then reopen it	Context carried across turns without naming the recording, and the conversation restored with its tool calls and artifacts.	FR-9
15	Re-index at a second depth	Two chunkings of one recording coexisting, and an added analyzer leaving the others’ work intact.	FR-11
16	Call the API directly	GET / <i>schema</i> describing the installation, and a search constructed from that response alone.	FR-12, NFR-8

Table 30: Demonstration sequence and requirement coverage. All twelve functional requirements are exercised.

Implemented and demonstrable in full: four boundary detectors, three chunking modes, six analyzers, twelve aggregators, five named vector spaces, twelve filters, three response detail levels, routed question answering with citations, nine agent tools, eight interface surfaces, and the documentation site with its API reference. Two items are stated as not implemented rather than demonstrated: cross-recording person linking, which is deferred because no fixture shares a subject across cameras, and live camera input, for which the pipeline is structured but no ingest path exists.

16 Product Demo: Correctness

Correctness is demonstrated on the same fixtures used throughout, so a result seen in the demonstration can be traced to a recorded acceptance case rather than taken on the day. Three claims are made and each is checked live.

Claim	How it is checked in the demonstration	Case
Retrieval is accurate	Five events are taken from the ground-truth sheet and queried in fresh wording. The correct moment ranks in the top three and its span lands within one chunk of the sheet's timecode.	AT-19
Answers are grounded	Every claim in an answer carries a timecode, and following the citation plays footage that supports it. A question the footage cannot answer is declined rather than answered plausibly.	AT-27, AT-29
The system is stable	The demonstration runs end to end on a deployed instance without a restart: concurrent ingestion and search, a failed source that reports its reason rather than hanging, and a conversation reopened after signing out and back in.	AT-11, AT-17, AT-31

Table 31: Correctness claims and their live checks.

Stability under the conditions that previously broke it is demonstrated explicitly, because those are the failures worth showing. A source that does not resolve to a video puts the recording into **failed** with its reason retained on the row and shown on the card, rather than leaving the workspace loading. A restart of the analysis service mid-ingest is detected by the client, which recovers the recording from its persisted record or fails the row so that re-indexing is offered, rather than polling a job that no longer exists. An unknown filter name is rejected with the nearest valid name rather than silently ignored. And a fabricated recording identifier supplied to the agent is filtered out by the project scoping function rather than answered from another operator's footage.

17 Product Performance

17.1 Precision

Retrieval precision is established by four measured separations rather than by a single accuracy figure, because the system makes four different decisions and each has its own operating point.

Two precision decisions are structural rather than threshold-based. The five named vector spaces raise precision on scoped queries, because a question about a person's appearance is matched against a short person description rather than competing with everything else recorded for that chunk. And the `people_count` filter exists because counting is what embeddings cannot do: a query for a mostly empty store retrieved the busiest chunk in the recording, since a description listing five people reads much the same to a vector as one listing two.

Decision	Measured separation	Operating point
Is this content present?	Content actually present scored ≥ 0.665 ; absent content ≤ 0.524 .	Score threshold 0.55 to 0.60, returned if the embedding model changes.
Is this the right moment?	Scored against the ground-truth sheet by the rank of the correct moment.	Correct moment within the top three, span within one chunk of the sheet.
Are these the same person?	Provably different people, co-visible in one chunk, reached 0.941; known-same pairs sat at 0.889 to 0.915. The bands overlap.	No threshold alone suffices. Merge above 0.88 under hard cannot-link constraints, with a distinctness gate at 0.80.
Which aggregate answers this?	Per-question score means shifted between 0.47 and 0.56, so no absolute cutoff works.	Relative selection: aggregates one standard deviation above the mean for that question, capped at three.

Table 32: Measured separations and the operating point each one sets.

17.2 Latency

Cold start is reported separately from all of the above, because a first request pays for model weight loading that no subsequent request repeats. All latency figures are taken warm, repeated ten times, and reported as the median together with the slowest of the ten.

Operation	Budget	Where the time goes
Search	$\leq 2\text{ s}$ (NFR-1)	Query embedding with a 33M-parameter model is a single short sentence on the GPU and is sub-millisecond. Search over an embedded Qdrant collection of the size this corpus produces is single-digit milliseconds. The budget is therefore dominated by HTTP round trips and browser render, not by retrieval.
Question	$\leq 15\text{ s}$ (NFR-1)	Routing is an embedding comparison against the aggregate hints and is negligible. One synthesis call to the language provider dominates and is the only network-bound step.
Playback	$\leq 2\text{ s}$ (NFR-4)	A clip is a range over an MP4 rather than a rendered file, so the first moment pays one media load and every later moment in the same reel pays a seek.
Progress refresh	$\leq 5\text{ s}$ (NFR-3)	The client polls only recordings actually in flight; a ready recording is not polled at all.
Ingestion	$\leq 3\times$ duration (NFR-2)	Dominated by vision-language calls, which are I/O bound and run across six worker threads. Local model passes are secondary: diarization on GPU runs far faster than real time, and frame reading is sequential rather than seeking, which was measured at approximately $22\times$ cheaper per frame.

Table 33: Latency budgets and their dominant costs.

17.3 Memory and Processor Usage

The analysis service was developed on an RTX 4060 with 8 GB of VRAM and 16 GB of system RAM, which is the reference configuration. The table below gives the GPU residency of each local model. Figures are the published or vendor-reported memory footprints for these model sizes; they are estimates for capacity planning rather than measurements taken on this machine.

Three properties keep this inside 8 GB. Models are loaded lazily, so a recording ingested without the diarization analyzer never loads pyannote at all. The pipeline is sequential by stage, so the peak is roughly the largest concurrent set rather than the sum of the table: chunking holds CLIP and whichever audio models the preset needs, analysis holds CLIP with YOLO or EasyOCR, and aggregation holds only GLiNER or the sentiment classifier. And frame sampling is serial because the CLIP model is shared on one GPU, so no second copy is ever allocated.

The cost control that matters most is not a resource limit but the shape of the pipeline. Cheap local detectors gate expensive remote calls, frames are deduplicated before analysis, analyzers are selected per upload, aggregators read records rather than video, and query time never reopens a video at all.

Model	Parameters	VRAM (est.)	When it is resident
OpenCLIP ViT-B/32	151M	290 MB weights, roughly 0.4 GB in use	Chunking (semantic drift) and frame deduplication in every frame-reading analyzer. The most frequently used local model.
YOLO11n	2.6M	Under 0.5 GB including CUDA context	The detection gate for the people and object analyzers, at 1 fps.
faster-whisper tiny	39M	Under 1 GB	Transcription. Runs on CTranslate2 rather than PyTorch, so its allocation is separate.
pyannote diarization	Segmentation plus embedding	1.5 to 2.5 GB, scaling with audio length	Speaker change detection and the diarization analyzer. The largest single item, and the one worth watching on long recordings.
EasyOCR (CRAFT detection)	20M	Roughly 0.7 to 1 GB transient at high input resolution	The text region gate, on OCR-selected frames only.
BGE-small-en-v1.5	33M	Roughly 64 MB in half precision	Indexing and every query embedding. Small enough to stay resident.
GLiNER small-v2.1	166M	Roughly 0.35 GB	The named entity aggregator only.
Sentiment classifier	125M	Roughly 0.5 GB	The sentiment aggregator only, on speech.

Table 34: Local model GPU residency. Values are published or vendor-reported footprints for these model sizes.

Resource	Behaviour
GPU utilisation	Bursty rather than sustained. Local inference is short, and the dominant ingest cost is waiting on the vision-language provider, during which the GPU is idle. Chunking is the most GPU-dense stage, since CLIP embeds one frame per second of footage.
CPU	Video decoding is the CPU-bound work and saturates roughly one core. Frames are read by walking the container forward with <code>grab()</code> and <code>retrieve()</code> rather than seeking; per-frame seeking forces the H.264 decoder to restart from a keyframe and was measured at approximately $22\times$ the cost per frame. The six worker threads issuing model calls are I/O bound and cost almost no CPU.
System RAM	16 GB is the stated requirement. The decoded mono waveform is the only whole-recording buffer: at 16 kHz float32 a 5-minute recording is approximately 19 MB and a 1-hour recording approximately 230 MB. Frames are handled one at a time rather than buffered whole.
Vector store memory	Five 384-dimensional float vectors per point is approximately 7.7 KB of vector data. A 5-minute recording producing around 30 chunks across four analyzers is roughly 120 points, under 1 MB. A corpus of a thousand such recordings is on the order of 1 GB, which embedded Qdrant holds comfortably.
Disk	The media cache holds one decoded copy of each ingested recording, keyed by content hash, and is regenerable. The record store holds one JSON document per recording and is the only expensive artefact. Model weights are downloaded once.
Client	Static-rendered Next.js pages with client-side caching and background revalidation. Opening a recording twice does not re-query the service, and playback is a single MP4 load per recording rather than one per clip.

Table 35: Resource behaviour across the system.

17.4 Scale Limits

Three limits are known and stated rather than discovered later. Qdrant payload indexes are inert in embedded mode, so filtering is correct but scans; running Qdrant as a server resolves this with no code change. Jobs are held in memory, so a restart discards job history while records and vectors survive on disk, and the client detects and recovers from the lost job. And diarization memory scales with audio length, so a multi-hour recording should be diarized over windows rather than in one pass.

18 UI Implementation

The interface as implemented is shown in Figure 6 and described in Sections 7 and 12. Three properties are claimed for it.

Property	How it is realised
Intuitive	One principle governs every screen: the operator never has to scrub. A search result, an answer citation and a clip row all resolve to a timecode that plays. Status is expressed in the vocabulary of the task, Uploading, Indexing and Ready, rather than in the vocabulary of the pipeline. A recording that cannot yet be searched says so instead of returning an empty list.
Smart	The agent layer removes the query language from the operator's job: a request in plain words becomes the right retrieval operation across nine tools, and context carries across turns without the recording being named again. The clip panel opens by itself when the results are moments. Progress is polled only for recordings actually in flight. Suggestion cards seed the first question so an empty workspace is not an empty prompt.
Changeable	The analyzer checklist and chunking selector are built from the service's capability endpoint, so a new analyzer appears in the interface with no client change (AT-10, AT-40). Interface primitives are shadow components vendored into the repository rather than installed from a package, so they are edited directly. Theming is Tailwind tokens rather than per-component styles, and the model selector is a registry entry, so adding a provider is one line.

Table 36: Interface properties and their implementation.

19 Documentation

Two mechanisms keep the documentation honest. The API reference is executable, because every endpoint it describes exists as a Postman request run against the service, so one that no longer behaves as documented fails a test rather than being quietly wrong (AT-41). And `GET /schema` describes the analyzers, filters and aggregators of the running instance, so a client can be built without reading the source, which AT-39 verifies by doing exactly that. Anything that cannot be checked this way, meaning the process record, the ground-truth sheets and the defect log, is kept beside the thing it describes: sheets with their fixtures, decisions with their code, and defect records with the test pass that found them.

Kind	Location	Contents
Coding documentation	<code>core/README.md</code> , <code>frontend/README.md</code> , and comments at the decision sites	How the engine works, how to add an analyzer or an aggregator, the environment that is verified to work, and a record of decisions that look odd but are deliberate, each with the measurement that produced it.
UI documentation	<code>frontend/content/docs/</code> , served at <code>/docs</code>	Workspace guides: projects, ingesting video, agent chat, clips and artifacts, and video details, with the screens as the operator meets them.
Control flow documentation	<code>/docs</code> concepts section and Section 6.1	The pipeline end to end: chunking, analyzers, aggregators, retrieval and question routing, plus the ingestion lifecycle and the job reconciliation that surrounds it.
Database interaction	Section 6.2 to 6.4, and the migrations in <code>frontend/lib/supabase/migrations/</code>	The four stores and why they are separate, the relational schema with its indexes and policies, the normalisation analysis, the vector point schema and filter declaration, and the content hash that joins all four.
API reference	<code>core/docs/ENDPOINTS.md</code> and <code>/docs</code>	Every endpoint with its parameters and response shape, mirrored by an executable Postman collection.
Operations	<code>core/docs/COMMANDS.md</code> , root <code>README.md</code>	Running, resetting, inspecting, the end-to-end walkthrough, and a symptom-to-cause troubleshooting table.
Design and test	<code>latex/</code>	The software design document, the acceptance test plan and cases, and this consolidated report.

Table 37: Documentation by kind.